Taha khan

C_C3_02

ML Lab - 2

In [ ]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import warnings
warnings.filterwarnings("ignore")
```

In [ ]:
```
df = pd.read_csv("california_housing.csv")
```

In [ ]:
```
df.head()
```

Out[5]:

	longitude	latitude	housingMedianAge	totalRooms	totalBedrooms	population	households	medianIncome	medianHouseValue
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0

In [ ]:
```
df.tail()
```

Out[6]:

	longitude	latitude	housingMedianAge	totalRooms	totalBedrooms	population	households	medianIncome	medianHouseValue
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0
20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   longitude         20640 non-null  float64
 1   latitude          20640 non-null  float64
 2   housingMedianAge  20640 non-null  float64
 3   totalRooms        20640 non-null  float64
 4   totalBedrooms     20640 non-null  float64
 5   population        20640 non-null  float64
 6   households        20640 non-null  float64
 7   medianIncome      20640 non-null  float64
 8   medianHouseValue  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

In [ ]:
```
df.describe()
```

Out[8]:

	longitude	latitude	housingMedianAge	totalRooms	totalBedrooms	population	households	medianIncome	medianHouseValue
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.898014	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.247906	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	295.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

In [ ]:
```
df.shape
```

Out[9]:
```
(20640, 9)
```

In [ ]:
```
df.columns
```

Out[10]:
```
Index(['longitude', 'latitude', 'housingMedianAge', 'totalRooms',
       'totalBedrooms', 'population', 'households', 'medianIncome',
       'medianHouseValue'],
      dtype='object')
```

In [ ]:
```
df.isnull().sum()
```

Out[11]:

	0
longitude	0
latitude	0
housingMedianAge	0
totalRooms	0
totalBedrooms	0
population	0
households	0
medianIncome	0
medianHouseValue	0

dtype: int64

In [ ]:
```
df.duplicated().sum()
```

Out[12]:
```
np.int64(0)
```

In [ ]:
```
df.drop_duplicates()
```

Out[13]:

	longitude	latitude	housingMedianAge	totalRooms	totalBedrooms	population	households	medianIncome	medianHouseValue
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0
...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0
20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0

20640 rows × 9 columns

In [ ]:
```
df.hist(figsize=(12,8))
plt.show
```

Out[14]:

matplotlib.pyplot.show
def show(*args, **kwargs) -> None

```
Display all open figures.

Parameters
----------
block : bool, optional
    Whether to wait for all figures to be closed before returning.

    If `True` block and run the GUI main loop until all figure windows
    are closed.

    If `False` ensure that all figure windows are displayed and return
    immediately.  In this case, you are responsible for ensuring
    that the event loop is running to have responsive figures.

    Defaults to True in non-interactive mode and to False in interactive
    mode (see `.pyplot.isinteractive`).

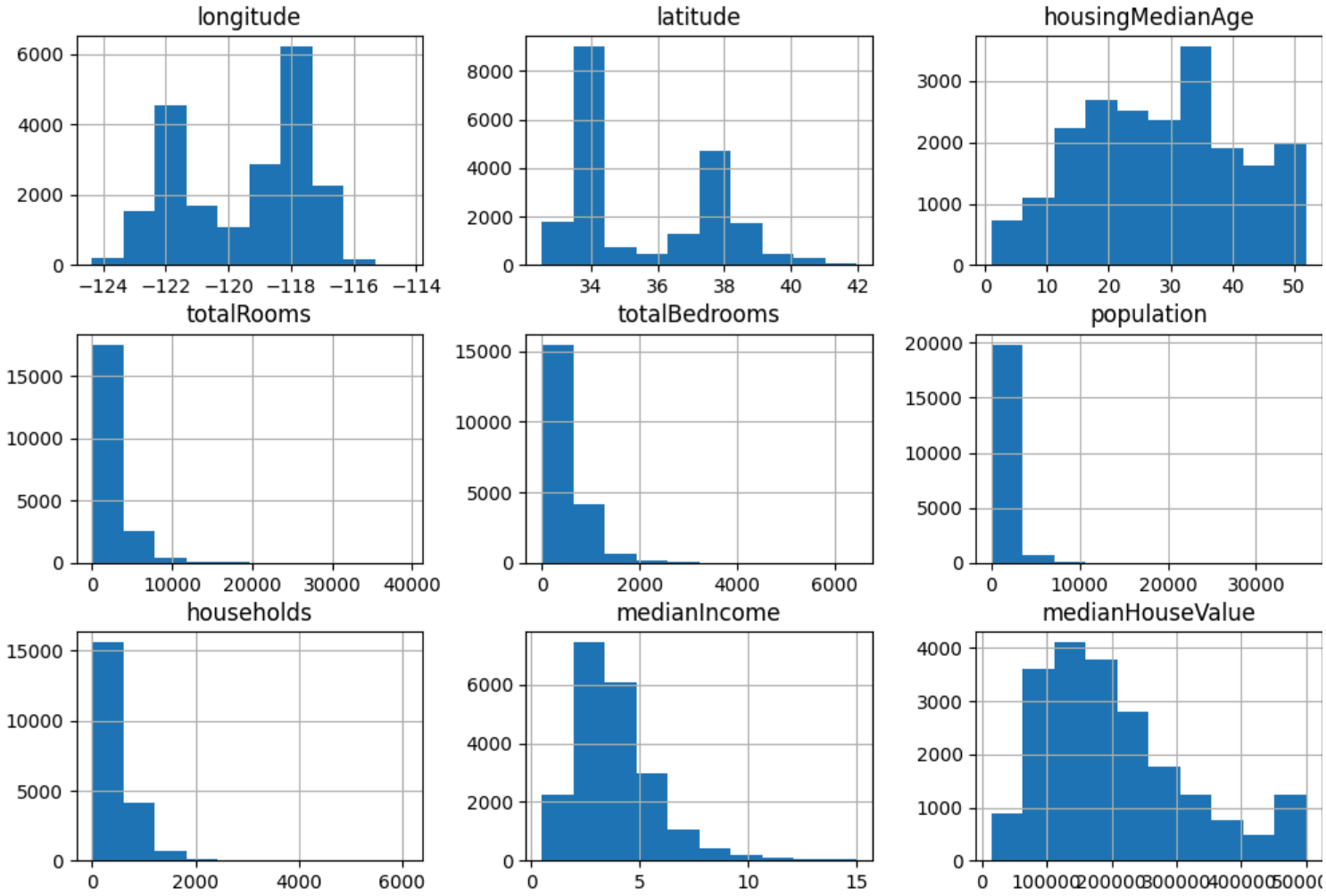
See Also
--------
ion : Enable interactive mode, which shows / updates the figure after
      every plotting command, so that calling ``show()`` is not necessary.
ioff : Disable interactive mode.
savefig : Save the figure to an image file instead of showing it on screen.

Notes
-----
**Saving figures to file and showing a window at the same time**

If you want an image file as well as a user interface window, use
`.pyplot.savefig` before `.pyplot.show`. At the end of (a blocking)
``show()`` the figure is closed and thus unregistered from pyplot. Calling
`.pyplot.savefig` afterwards would save a new and thus empty figure. This
limitation of command order does not apply if the show is non-blocking or
if you keep a reference to the figure and use `.Figure.savefig`.

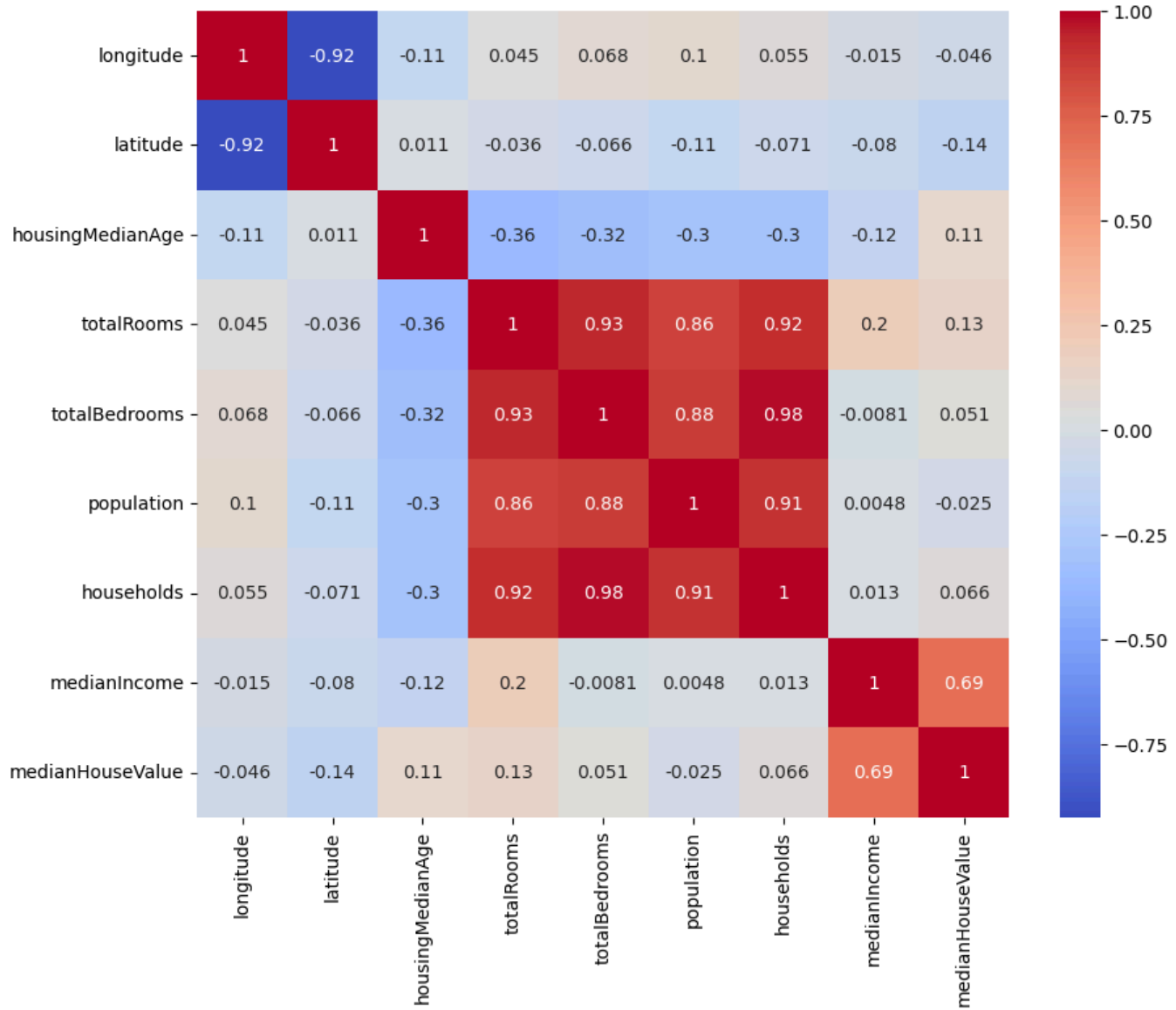
**Auto-show in jupyter notebooks**

The jupyter backends (activated via ``%matplotlib inline``,
``%matplotlib notebook``, or ``%matplotlib widget``), call ``show()`` at
the end of every cell by default. Thus, you usually don't have to call it
explicitly there.
```


longitude
latitude
housingMedianAge
totalRooms
totalBedrooms
population
households
medianIncome
medianHouseValue

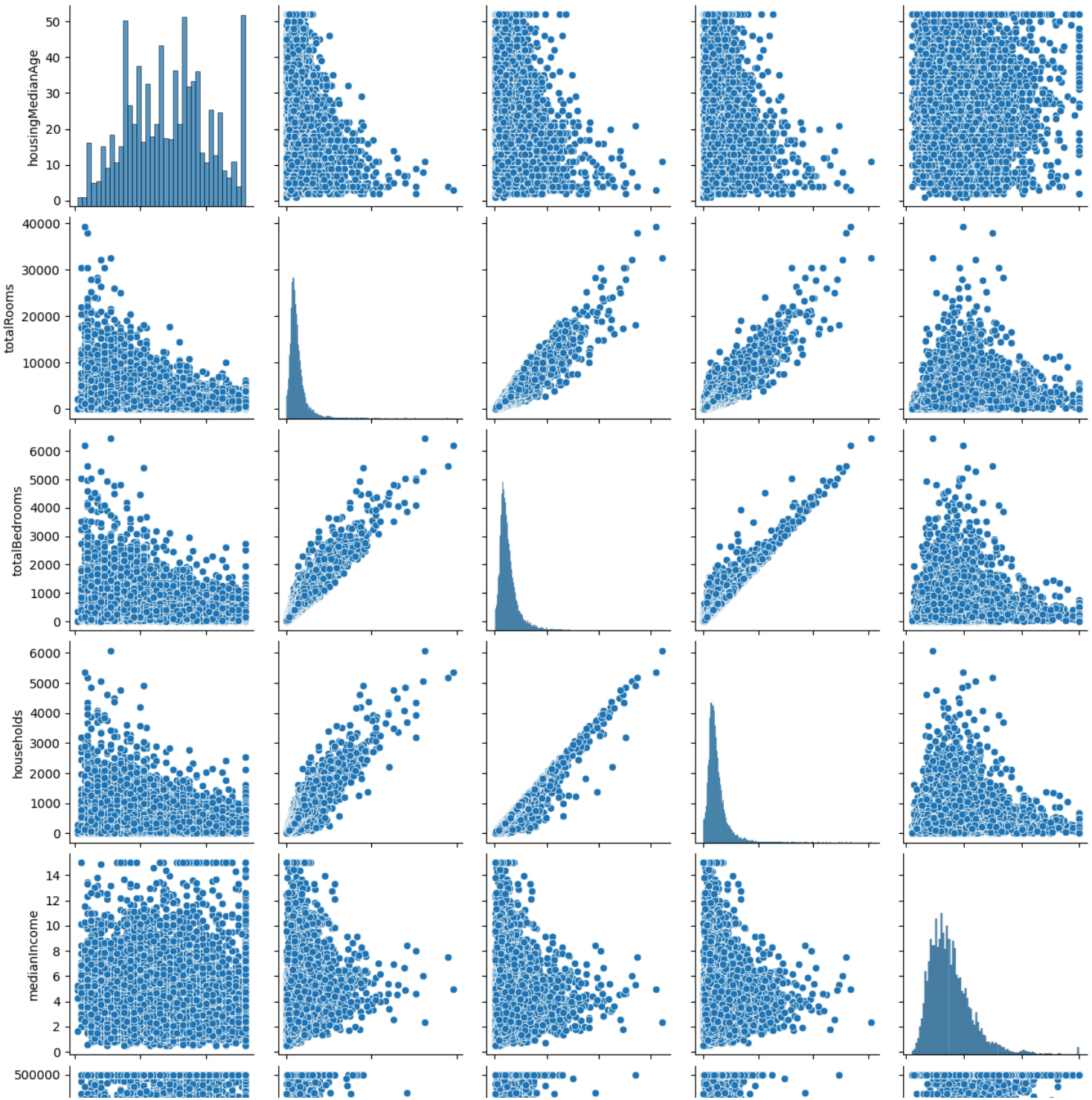
In [ ]:
```
plt.figure(figsize=(10,8))

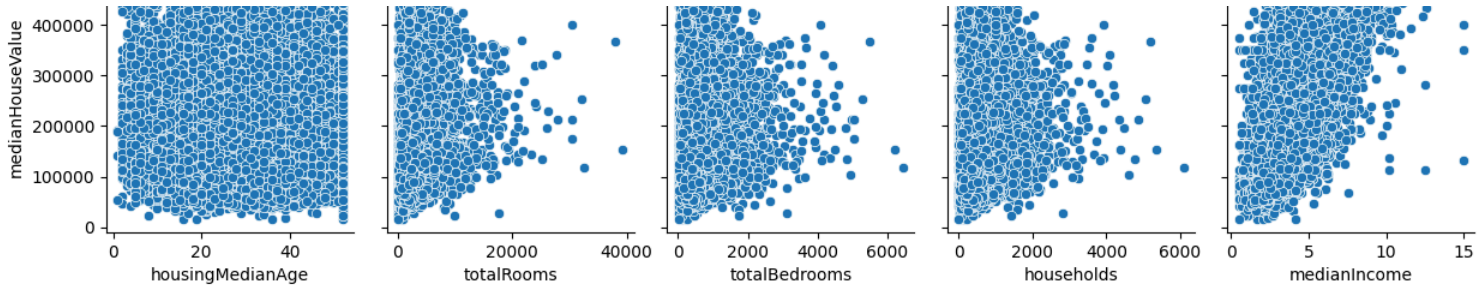
sns.heatmap(df.corr(),annot=True,cmap="coolwarm")
plt.show()
```



In [ ]:

```
sns.pairplot(df[['housingMedianAge', 'totalRooms',
       'totalBedrooms','households', 'medianIncome',
       'medianHouseValue']])
plt.show()
```


housingMedianAge
totalRooms
totalBedrooms
households
medianIncome
0
10
20
30
40
50
0
10000
20000
30000
40000
0
1000
2000
3000
4000
5000
6000
0
1000
2000
3000
4000
5000
6000
0
2
4
6
8
10
12
14
500000



In [ ]:
```
df.columns
```

Out[17]:
```
Index(['longitude', 'latitude', 'housingMedianAge', 'totalRooms',
       'totalBedrooms', 'population', 'households', 'medianIncome',
       'medianHouseValue'],
      dtype='object')
```

Simple Linear Regression

In [ ]:
```
X = df[['medianIncome']]
y = df['medianHouseValue']
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.20,
    random_state = 1300
)
```

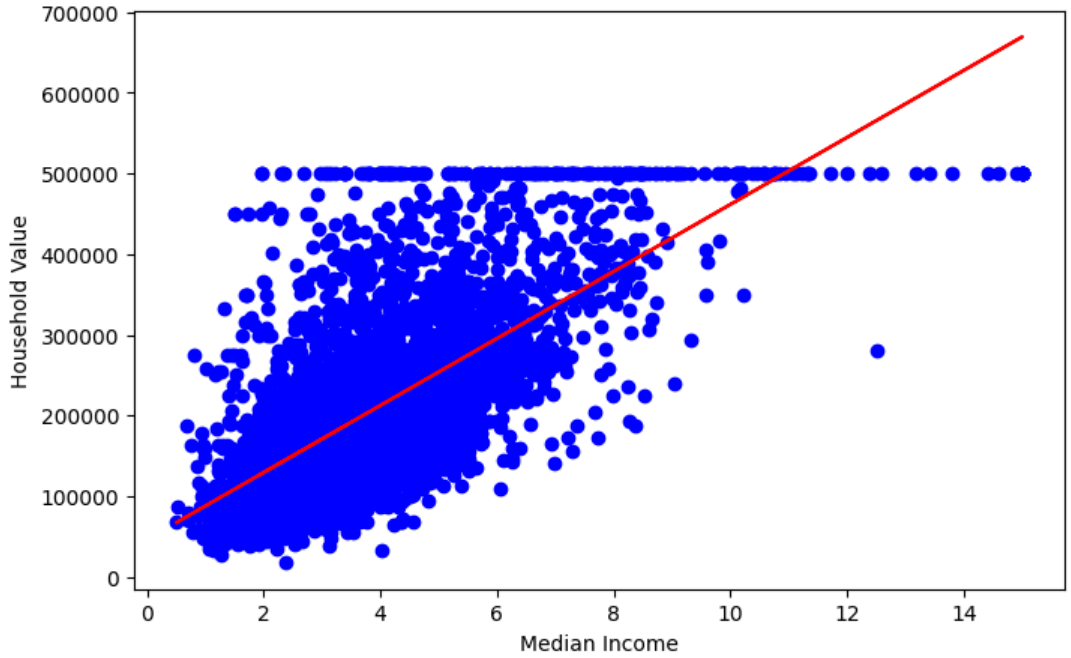
In [ ]:
```
slr = LinearRegression()
slr.fit(X_train,y_train)
```

Out[20]:
```
LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
y_pred = slr.predict(X_test)
```

In [ ]:
```
plt.figure(figsize=(8,5))
plt.scatter(X_test,y_test,color="blue")
plt.plot(X_test,y_pred,color="red")
plt.xlabel("Median Income")
plt.ylabel("Household Value")
plt.show()
```



In [ ]:
```
mae = mean_absolute_error(y_test,y_pred)
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)
print("MAE: ",mae)
print("MSE: ",mse)
print("RMSE: ",rmse)
print("R2-Score: ",r2)
```

```
MAE:  62606.40035097627
MSE:  6898954157.023464
RMSE:  83059.94315567201
R2-Score:  0.46801312690949415
```

Multiple Linear Regression

In [ ]:
```
df.columns
```

Out[24]:
```
Index(['longitude', 'latitude', 'housingMedianAge', 'totalRooms',
       'totalBedrooms', 'population', 'households', 'medianIncome',
       'medianHouseValue'],
      dtype='object')
```

In [ ]:
```
X = df.drop("medianHouseValue",axis=1)
y = df["medianHouseValue"]
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.20,
    random_state = 1300
)
```

In [ ]:
```
mlr = LinearRegression()
mlr.fit(X_train,y_train)
```

Out[27]:
```
LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
y_pred = mlr.predict(X_test)
```

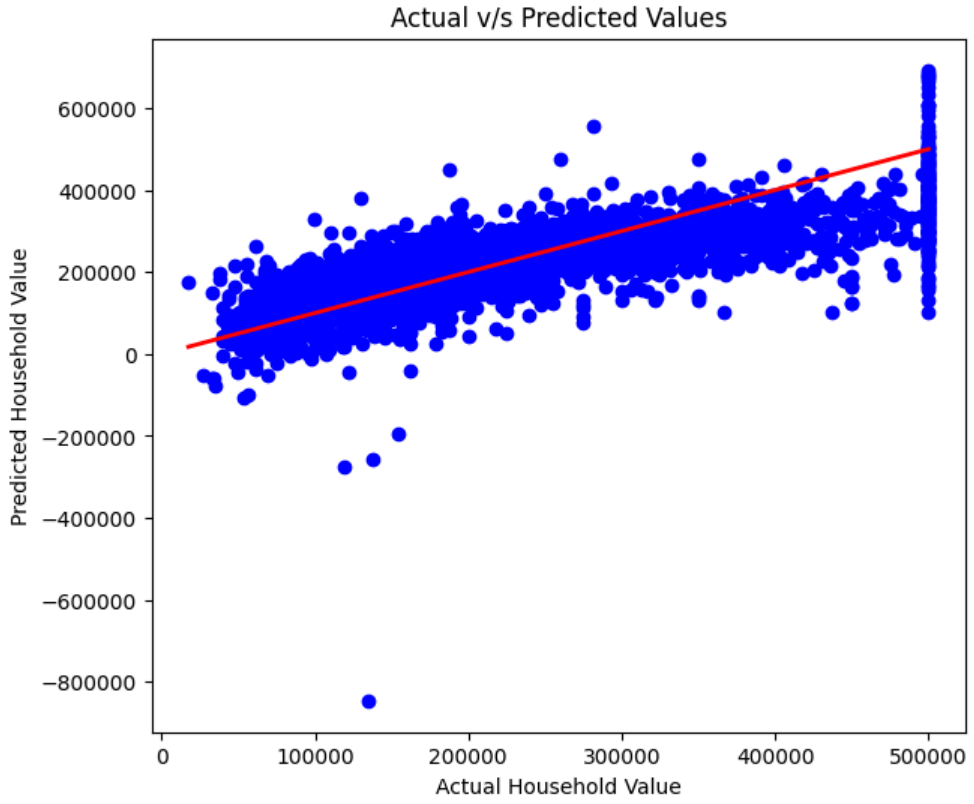
In [ ]:
```
mae = mean_absolute_error(y_test,y_pred)
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)
print("MAE: ",mae)
print("MSE: ",mse)
print("RMSE: ",rmse)
print("R2-Score: ",r2)
```

```
MAE:  50765.89530903358
MSE:  5048101585.288719
RMSE:  71049.99356290414
R2-Score:  0.610734653937803
```

In [ ]:
```
plt.figure(figsize=(7,6))
plt.scatter(y_test,y_pred,color="blue")

plt.plot([y_test.min(),y_test.max()],
         [y_test.min(),y_test.max()],
         color="red",linewidth=2)

plt.xlabel("Actual Household Value")
plt.ylabel("Predicted Household Value")
plt.title("Actual v/s Predicted Values")
plt.show()
```



Hyperparameters Tunning

In [ ]:
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error , mean_squared_error , r2_score
import numpy as np
```

In [ ]:
```
lr = LinearRegression()
```

In [ ]:
```
lr_params = {
    'fit_intercept' : [True,False],
    'positive' : [True,False]
}
```

In [ ]:
```
grid_search = GridSearchCV(
    estimator = lr,
    param_grid = lr_params,
    cv = 5,
    scoring = 'r2',
    n_jobs = -1
)
```

In [ ]:
```
grid_search.fit(X_train, y_train)
```

Out[35]:
```
GridSearchCV(cv=5, estimator=LinearRegression(), n_jobs=-1,
             param_grid={'fit_intercept': [True, False],
                         'positive': [True, False]},
             scoring='r2')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
print("Best Parameters :")
print(grid_search.best_score_)
```

```
Best Parameters :
0.6419516687624979
```

In [ ]:
```
best_model=grid_search.best_estimator_
```

In [ ]:
```
LinearRegression(
    fit_intercept=True,
    positive=True
)
```

Out[38]:
```
LinearRegression(positive=True)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
rmse=np.sqrt(mean_squared_error(y_test,y_pred))
mae=mean_absolute_error(y_test,y_pred)
r2=r2_score(y_test,y_pred)
mse=mean_squared_error(y_test,y_pred)
print("MSE : ",mse)
print("RMSE : ",rmse)
print("MAE : ",mae)
print("R2 : ",r2)
```

```
MSE :  5048101585.288719
RMSE :  71049.99356290414
MAE :  50765.89530903358
R2 :  0.610734653937803
```

In [ ]:
```
print(best_model)
```

```
LinearRegression()
```

In [ ]:
```
result=grid
```

ridge regression

In [ ]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
import numpy as np
```

In [ ]:
```
from sklearn.linear_model import Ridge
ridge=Ridge()
ridge.fit(X_train,y_train)
ridge_pred=ridge.predict(X_test)
print("default ridge r2:",r2_score(y_test,ridge_pred))
```

```
default ridge r2: 0.6107358522376658
```

In [ ]:
```
param_grid={'alpha':[0.001,0.01,0.1,1,10,100,1000]}
```

In [ ]:
```
grid_ridge=GridSearchCV(estimator=ridge,param_grid=param_grid,scoring='r2',cv=5)
```

In [ ]:
```
grid_ridge.fit(X_train,y_train)
```

Out[56]:
```
GridSearchCV(cv=5, estimator=Ridge(),
             param_grid={'alpha': [0.001, 0.01, 0.1, 1, 10, 100, 1000]},
             scoring='r2')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
print("Best Alpha:",grid_ridge.best_params_)
```

```
Best Alpha: {'alpha': 1}
```

In [ ]:
```
print("Best Alpha:",grid_ridge.best_score_)
```

```
Best Alpha: 0.6419517144527092
```

In [ ]:
```
best_ridge=grid_ridge.best_estimator_
ridge_pred=best_ridge.predict(X_test)
```

In [ ]:
```
print("MAE:",mean_absolute_error(y_test,ridge_pred))
print("MSE:",mean_squared_error(y_test,ridge_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test,ridge_pred)))
print("R2:",r2_score(y_test,ridge_pred))
```

```
MAE: 50765.7296331818
MSE: 5048086045.401857
RMSE: 71049.88420400034
R2: 0.6107358522376658
```

Lasso Regression

In [ ]:
```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV
param_grid={'alpha':[0.001,0.01,0.1,1,10]}
```

In [ ]:
```
grid_lasso=GridSearchCV(Lasso(max_iter=5000),param_grid,cv=5,scoring='r2')
```

In [ ]:
```
grid_lasso=GridSearchCV(Lasso(max_iter=5000),param_grid,cv=5,scoring='r2')
grid_lasso.fit(X_train,y_train)
```

Out[65]:
```
GridSearchCV(cv=5, estimator=Lasso(max_iter=5000),
             param_grid={'alpha': [0.001, 0.01, 0.1, 1, 10]}, scoring='r2')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
print(grid_lasso.best_params_)
```

```
{'alpha': 10}
```

In [ ]:
```
lasso_pred=grid_lasso.predict(X_test)
print(r2_score(y_test,lasso_pred))
```

```
0.6107387331558796
```

In [ ]: